Requirements

Requirements

Requirements are the *foundation* of any successful project

They define

> what the project is supposed to do, how it should perform, and what constraints it must operate within.

One of the most important aspects of project management is gathering and managing requirements effectively

It's also *surprisingly* difficult

Formally

> Requirements are a specification of what should be implemented. They are descriptions of how the system should behave, or of a system property or attribute. They may be a constraint on the development process of the system.

Mini exercise: Building a shelf

Write down the requirements for building a shelf, on a notepad, a piece of paper, or just in your head

After 5 minutes

I'll ask a random selection to give me *one* requirement

Why are requirements important

> The **Primary** and most important reason is that they are the basis for *all* project planning and execution.

For smaller personal projects, requirements may be informal and implicit,

But for larger projects, they **must** be formal and explicit

You **cannot** manage multiple people working on a project without clear requirements

This applies to *any* project, whether it's building a shelf, writing a thesis, or developing software

Levels and types of requirements

Before getting any requirements, it's useful to have a *standard* way of categorizing them

The most common and *broad* categories of requirements are

- *Functional* requirements - what the system should do
- *Non-functional* requirements - how the system should do it, or constraints on the system

	Functional	Non Functional
Examples	User auth, data input/outupt, transactions	Scalability, security, speed, reliability
Focus	Business requirements	User experience

Levels and types of requirements

There are also a bunch of different categorizations, some of them will be more useful than others depending on the project

Term	Definition
Business requirement	A high-level business objective of the organization
Constraint	A restriction that is imposed on the choices available
Feature	One or more logically related system capabilities
Quality Attribute	Describes a service or performance characteristic
System requirement	A top-level requirement which contains subsystems
User requirement	A goal or task that a type of user must be able to do

Mini exercise: Categorizing requirements

With our shelf example, categorize each requirement

Note that many of these requirements are *not* mutually exclusive, and some may fit into multiple categories

Break

How do you get requirements

This is usually the step called *elicitation*, which is the process of gathering requirements from stakeholders

Primarily it deals with

- Identifying the product's expected *user classes* and other *stakeholders*
- Understanding the users' *needs and goals*
- Understanding the users' *environment* and constraints
- Working with individuals who *represent* each user class

How do you get requirements

There are a bunch of different techniques for eliciting requirements, some common ones are

1. Interviews
2. Workshops
3. Focus groups
4. Observation
5. Questionnaires
6. Prototyping

How do you get requirements: Interviews

Direct discussion with stakeholders to understand their goals and needs.

- **Pros**: Higher detail, allows for follow-up questions
- **Cons**: Time-consuming, results depend on interviewer skill

> *Example*: Asking a store manager how they currently track inventory to design an automated system.

How do you get requirements: Workshops

Structured meetings with a group of stakeholders to define requirements together.

- **Pros**: Fast consensus, identifies conflicting requirements early.
- **Cons**: Can be dominated by "loud" voices

> *Example*: A 4-hour session with developers, marketing, and sales to define the core features of a new mobile app.

How do you get requirements: Focus groups

Representative users discuss the product in an informal setting.

- **Pros**: Reveals user attitudes and expectations.
- **Cons**: Groupthink can bias results, not suitable for technical requirements.

> *Example*: Showing a UI mockup to five frequent shoppers to see how they feel about the checkout flow.

How do you get requirements: Observation

Watching users perform their tasks in their actual work environment.

- **Pros**: Identifies "unspoken" requirements and real-world constraints.
- **Cons**: Users may act differently when watched, time-intensive.

> *Example*: Sitting with a bank teller for a day to see how they handle customer deposits and withdrawals.

How do you get requirements: Questionnaires

Surveys sent to a large number of people to gather data.

- **Pros**: Cheap, easy to scale, good for quantitative data.
- **Cons**: No follow-up possible, low response rates, can be biased.

> *Example*: Sending a Google Form to 500 students to ask what features they want in a campus map app.

How do you get requirements: Prototyping

Building a "lite" version of the system to get feedback.

- **Pros**: Users find it easier to provide feedback on something "real."
- **Cons**: Can be expensive, users might focus too much on visuals over logic.

> *Example*: Creating a clickable Figma mockup of a website before writing any code.

Requirements Analysis

After gathering requirements, you need to analyze them to ensure they are clear, complete, and consistent

Some basic things you need to do are

- *Distinguish* task goals from functional requirements, quality expectations, business rules, user needs, etc
- *Decompose* high-level requirements into more detailed ones
- *Deriving* requirements from other requirements
- Negotiating implementation *priorities*
- Finding *gaps* in the requirements

Validation

After analyzing the requirements, you need to validate them to ensure they are correct and meet the needs of the stakeholders

This is usually done through *reviews* and *inspections*, where stakeholders review the requirements and provide feedback

Break

Activity: Requirements

Group yourselves into groups of 3-4 people (write names down on paper)

Create a list of (categorized) requirements for a proposed simple system

- so create a list of requirements
- say what categories they fall into
- explain why you have these requirements
- and why you categorized them the way you did

Present your requirements to the class

- You have 60 minutes
- At least 10 requirements
- Everyone must present
- No powerpoint (just a doc of requirements)
- I'll ask questions about your requirements, so be prepared

Available systems

1. *local coffee shop* (Online ordering and loyalty system)
2. *small community library* (Digital catalog and book loan management)
3. *boutique gym* (Membership portal and class booking)
4. *veterinary clinic* (Appointment scheduling and patient records)
5. *plant nursery* (Online store and plant care guide integration)
6. *local food bank* (Donation tracking and volunteer scheduling)
7. *independent bookstore* (Inventory management and event registration)
8. *coworking space* (Desk booking and member community portal)

Work time

Where things can go wrong

Any system, large or small, has requirements

The more requirements there are, the more likely someone will forget one, or misunderstand one, or miscommunicate one

1. insufficient user involvement
2. innacurate planning
3. creeping user requirements
4. ambiguous requirements
5. gold plating
6. overlooked stakeholders

1. Insufficient User Involvement

1. Customers often don't understand *why it's essential* to work hard on requirements
2. Developers might not emphasize user involvement because they might think they *already* understand
3. In some cases it's *difficult to get access* to people who will actually use the product

These problems lead to **breaking** requirements that require *reworking* and *delay* completion

Simply, communication is *difficult*, *time-consuming*, and prone to *misunderstanding*

2. Inaccurate Planning

> "Here's my idea for a new product; when will you be done?"

No one should answer this question until more is known about the problem being discussed.

Vague, poorly understood requirements lead to overly optimistic estimates, which come back to bite you later

A quick guess sounds **a lot like a commitment**.

The top contributors to poor software cost estimation are

- frequent *requirements changes*,
- *missing* requirements,
- *insufficient* communication with users,
- *poor* specification of requirements,
- and *insufficient* requirements analysis.

Estimating project effort and duration based on requirements means that you **need** to know something about the *size* of your requirements

3. Creeping User Requirements

As requirements evolve during development, projects often **exceed their planned schedules and budgets** (which are nearly always *too optimistic* anyway).

To manage *scope creep*, begin with

- a clear statement of the project's business objectives,
- strategic vision,
- scope,
- limitations,
- and success criteria.

Evaluate *all proposed new features* or requirements changes against this reference.

Requirements **will** change and grow.

The project manager should build *contingency* buffers into schedules so the first new requirement that comes along doesn't derail the schedule.

Case Study: LocalBites

A restaurant discovery and ordering app. Assume that you are already in the middle of development

A major stakeholder watches attends a seminar and gets inspired. They request a new feature:

> "An AI-powered, weekly subscription meal-planner that automatically orders healthy dinners for users based on their fitness goals."

- *Business Objective*: Increase online order volume for local restaurants by 20% within the first six months of launch.
- *Vision*: To be the most reliable, community-driven, on-demand food delivery platform for local neighborhoods.
- *Scope*: Core ordering system, real-time driver tracking, and standard one-time checkout.
- *Limitation*: Launch fixed in 4 weeks; remaining budget is $15,000; technical stack is optimized for basic data retrieval, not machine learning.
- *Criteria*: 5,000 active users in Month 1; less than a 2% order error rate.

Activity: Scope Creep

Individually, discuss how you would respond to the stakeholder's request for the AI-powered meal planner feature.

With the format:

1. **Initial Reaction**: What is your immediate response to the request?
2. **Evaluation**: How does this new requirement align with the project's business objectives, vision, scope, limitations, and success criteria?
3. **Decision**: Would you approve, reject, or defer this requirement? Justify

Submit as an essay in NEO

- be prepared to answer questions about your decision

4. Ambiguous Requirements

One symptom of ambiguity in requirements is that a reader can interpret a requirement statement in *several ways*. Another sign is that multiple readers of a requirement arrive at *different understandings* of what it means.

Ambiguity leads to *different expectations* on the part of various stakeholders.

- Some of them are then *surprised* at whatever is delivered.

Ambiguous requirements **cause wasted time**

- when developers *implement a solution for the wrong problem*.
- Testers who *expect the product to behave differently* from what the developers built

Collaborative validation encourages discussions and clarifies requirements as a group in a *workshop setting*.

Writing *tests against the requirements* and *building prototypes* are other ways to discover ambiguities.

> "The system must send a notification immediately when a high-value transaction occurs."

5. Gold Plating

Gold plating takes place when a developer adds functionality that **wasn't** in the requirements but which the developer believes "*the users are just going to love.*"

If users **don't care** about this functionality, the time spent implementing it is wasted.

Rather than simply inserting new features, developers should *present* stakeholders with creative ideas for their consideration.

Developers should strive for *leanness and simplicity*, *not going beyond* what stakeholders request **without their approval**,

Customers sometimes request certain features or elaborate user interfaces that *look attractive* but *add little value* to the product.

Everything you build *costs time and money*, so you need to maximize the delivered value.

To reduce the threat of gold plating, *trace each bit of functionality back to its origin* so everyone knows why it's included.

6. Overlooked Stakeholders

Most products have *several groups of users* who might

- use different subsets of features,
- have different frequencies of use,
- or have varying levels of experience.

Besides obvious users,

- think about *maintenance* and *field support* staff who have their own requirements, both functional and nonfunctional.
- People who have to *convert data from a legacy system* will have *transition requirements* that don't affect the ultimate product software but that certainly influence solution success.
- You might have stakeholders who *don't even know the project exists*, such as *government agencies* that mandate standards that affect your system, yet you need to know about them and their influence on the project.

Benefits of a high-quality requirements process

Investing in good requirements will *virtually always return* more than it costs.

You're going to get customer input eventually, it's cheaper to reach this understanding **before** you build the product than after delivery.

Even if you can't quantify all of these benefits, they are real

- Fewer *defects* in requirements and in the delivered product.
- Reduced development *rework*.
- *Faster* development and delivery.
- Fewer *unnecessary and unused* features.
- Fewer *miscommunications*.
- Reduced *scope creep*.
- Higher *customer* and *team member* satisfaction.